Bridging the Accuracy Gap: Witness-Cosigned Hash Chains for FDA-Compliant AI Agent Audit Trails

vaos-kernel Contributors

April 3, 2026

Abstract

Hash-chained audit trails can prove that logged entries were not modified, but categorically cannot prove that logged actions actually occurred. In regulated settings (pharmaceutical manufacturing, clinical trials, medical devices), this "accuracy gap" is the difference between a technically correct audit log and a legally submissible one under 21 CFR Part 11. We propose **witness cosigning**—an architectural extension where independent observers countersign agent actions before they are chained—and analyze its security, overhead, and regulatory implications. This is the first paper to formally characterize the accuracy gap in hash-chained audit trails for FDA-regulated AI agent systems, propose witness cosigning as a constructive solution, and provide a concrete regulatory gap analysis mapping technical mechanisms to 21 CFR Part 11 requirements.

1 Introduction

1.1 Core Idea

This paper addresses a concrete problem at the intersection of AI agent governance and FDA regulatory compliance: hash-chained audit trails (as in vaos-kernel, Certificate Transparency, and similar systems) can prove that logged entries were not modified, but categorically cannot prove that logged actions actually occurred. In regulated settings (pharmaceutical manufacturing, clinical trials, medical devices), this "accuracy gap" is the difference between a technically correct audit log and a legally submissible one under 21 CFR Part 11. We propose **witness cosigning**—an architectural extension where independent observers countersign agent actions before they are chained—and analyze its security, overhead, and regulatory implications.

1.2 Contribution 1: Honest Classification of ALCOA+ Properties Against Hash Chains

We provide a straightforward classification of the 9 ALCOA+ attributes against hash-chained audit, comparing vaos-kernel with Certificate Transparency (RFC 6962/Trillian) and Hyperledger Fabric's block-based audit. The classification is not claimed as a theorem—it is a structured engineering analysis:

- **Properties hash chains can enforce:** Consistent (append-only), Original (tamper detection)—these reduce to collision resistance of the chaining hash.
- **Properties the architecture supports under trust assumptions:** Attributable (assuming credential security), Contemporaneous (assuming server clock integrity), Complete (assuming `Record()` is the only entry path), Enduring (assuming correct database administration).
- **Properties no audit log can enforce alone:** Accurate (logging X doesn't prove X happened), Legible (machine-readable ≠ human-readable across decades), Available (a single database instance is not "available" under FDA's meaning).

We ground this with a concrete threat: a database-level attacker who compromises the Postgres instance storing `alcoa_actions` can rewrite the entire chain from the known genesis value `vaos-kernel-genesis-0000...0000`. Without an external anchor, the chain provides tamper *evidence* only against attackers who cannot write to storage. We compare how CT addresses this via Merkle tree heads published to an independent gossip network, and argue that analogous anchoring is needed for regulated agent audit.

1.3 Contribution 2: Witness Cosigning Protocol

The paper's primary constructive contribution is a **witness cosigning protocol** that bridges the accuracy gap for the Accurate property and strengthens Tier 1 guarantees via external anchoring.

Protocol:

1. Agent declares intent → produces `AuditEntry` with fields populated but no hash computed
2. Entry is dispatched to W independent witnesses (human supervisors, separate services, hardware security modules, or sensor systems depending on domain)
3. Each witness independently verifies the action was performed (mechanism varies by domain—see instantiation below)
4. Witness returns signature σ_w over `(canonical_fields ‖ witness_id ‖ witness_timestamp)`
5. `AuditEntry.Attestation` = Aggregate($\sigma_{w_1}, \ldots, \sigma_{w_K}$) where $K \geq$ threshold T
6. Only after threshold is met does `Record()` compute $H_n = \text{BLAKE2b}(\text{canonical\_fields}_n \| \text{signatures}_n \| H_{n-1})$ and append to chain

Key properties:

- The witness signatures are included in the hash input, so modifying or removing them breaks the chain—making cosigning a Tier 1 (cryptographically guaranteed) property of the resulting chain.
- The most recent chain hash, when published to an external medium (e.g., a notarization service, HSM, or independent log), anchors the chain against database-level compromise.
- Accuracy is elevated from "the agent claims X" to "the agent and T independent observers attest X." Under an (n, t)-threshold trust model, accuracy holds if fewer than t witnesses are compromised.

Domain instantiations:

- **Clinical trial AI:** Clinician operator countersigns via FIDO2/WebAuthn (satisfying §11.200's two-component signature requirement—something you have + biometric/PIN)
- **Lab automation:** PLC or sensor provides an independent reading hash, cosigning the agent's action
- **Pharmaceutical manufacturing:** DCS/SCADA system countersigns, with the most recent chain hash anchored to a plant historian system (addressing the external anchor problem)

We provide a formal security game for the (n, t)-threshold accuracy property and show that breaking accuracy requires compromising $\geq t$ witnesses—a fundamentally different (and stronger) trust assumption than the unattested chain.

1.4 Contribution 3: Quantitative Overhead of Witness Cosigning

Using vaos-kernel's existing benchmarks as baseline, we model overhead for the witness protocol:

Key finding: Witness cosigning adds sub-millisecond overhead with a local HSM witness—negligible compared to the 622ms Postgres write latency that already dominates. The protocol's cost is dominated by network RTT to remote witnesses, not cryptographic operations. For pharmaceutical manufacturing where a plant-floor HSM is co-located, witness cosigning is effectively free relative to existing constraints.

Configuration	Estimated p99 Latency	Regulatory Viability
Sync + ALCOA+ (current, in-memory)	1.69 ms	Baseline
+ 1 local witness (HSM, same rack)	∼2.2 ms (+0.5ms)	Viable for real-time mfg
+ 1 remote witness (same datacenter)	∼12–50 ms	Viable for clinical trials
+ 3-of-5 threshold witnesses (fanout)	∼60–80 ms	Viable for batch record systems
Sync + Postgres (current)	622 ms	Binding constraint
+ 1 local witness + Postgres	∼623 ms	Witness adds negligible overhead

Table 1: Witness cosigning overhead comparison

1.5 Contribution 4: Regulatory Gap Analysis with Constructive Resolution

We identify 6 specific gaps between vaos-kernel (and IETF draft-goswami-agentic-jwt-00) and 21 CFR Part 11 / EU Annex 11, and for each gap, state whether witness cosigning resolves it or whether organizational controls are required:

Gap	CFR Reference	Resolved by Witness Cosigning?
No validation lifecycle documentation	§11.10(a)	No—organizational control required
No export-to-standard-format mechanism	§11.10(b)	No—engineering work required
60-second JWT $\neq$ full retention period	§11.10(c)	Partially
No signature meaning attribution	§11.50	**Yes**
Intent fingerprint $\neq$ electronic signature	§11.70	**Yes**
No two-component signing	§11.200	**Yes**

Table 2: Regulatory gap analysis

The honest demarcation between technical and organizational gaps is itself a practical contribution: practitioners deploying AI agents in GxP environments can use this as a compliance roadmap.

1.6 Contribution 5: Comparison with Certificate Transparency and Other Systems

We generalize the analysis by comparing witness cosigning against existing approaches:

- **Certificate Transparency (RFC 6962):** Uses Merkle tree heads gossiped across independent monitors for tamper detection. Monitors detect misissuance but do not *attest* that the certificate was legitimately issued—they only verify log inclusion. Witness cosigning provides a stronger guarantee: active attestation of the action's accuracy, not passive monitoring of log consistency.
- **Trillian (production-grade CT implementation):** Provides sparse Merkle trees with efficient inclusion proofs but no mechanism for third-party action attestation. The accuracy gap exists in Trillian as well.
- **Hyperledger Fabric:** Endorsement policies require specified peers to sign transactions before commitment—architecturally similar to witness cosigning but designed for business logic validation, not regulatory data integrity. We argue that Fabric endorsement policies could be adapted for ALCOA+ compliance.
- **W3C Verifiable Credentials / C2PA:** Address provenance and authenticity for digital media and identity but do not engage with ALCOA+ or FDA regulatory requirements.

The comparison shows that the accuracy gap is a shared limitation across all major append-only log systems, and witness cosigning is a generalizable solution, not a vaos-kernel-specific hack.

1.7 Novelty Statement

This is the first paper to (1) identify and formally characterize the accuracy gap in hash-chained audit trails for FDA-regulated AI agent systems, (2) propose witness cosigning as a constructive solution that elevates accuracy from a Tier 3 (unenforceable) property to a Tier 1 (cryptographically guaranteed) property under explicit trust assumptions, (3) address the external anchor problem for chain integrity against database-level attackers, and (4) provide a concrete regulatory gap analysis mapping technical mechanisms to 21 CFR Part 11 requirements with honest demarcation between what code solves and what process must solve. The contribution is both analytical (the classification) and constructive (the protocol), positioned as a 6–8 page workshop or conference paper.

Recommended Venues: IEEE EuroS&P Workshop on AI Safety, ACM CCS Workshop on Decentralized Identity, USENIX Security short paper, or a regulatory informatics venue (JAMIA, with clinical co-author).

2 Methodology

The research proceeds in five methodological phases, each corresponding to one of the paper's contributions. The phases are sequential for Phases 1–3 (each depends on the prior), while Phases 4 and 5 can proceed in parallel after Phase 1 is complete.

2.1 Phase 1: ALCOA+ Property Classification Against Hash-Chained Audit

2.1.1 Step 1.1: Formal Specification of the vaos-kernel Audit Mechanics

Begin by extracting the exact specification of the hash chain from the source code. Clone the repository at the latest tagged release. Read `internal/audit/ledger.go` and produce a formal pseudocode description of the `Record()` function, capturing:

- The canonical ordering of fields used in the hash input. The source code computes $H_n = \text{BLAKE2b}(\text{canonical\_fields}_n \| H_n$ but the exact serialization order of fields must be extracted and documented verbatim.
- The genesis hash constant: `vaos-kernel-genesis-0000...0000`.
- The field validation logic: which fields are required (`AgentID`, `Component`, `Action`), which are optional (`Details`, `Status`), and how the timestamp is auto-generated (UTC, what precision).
- The `VerifyChain()` and `Replay()` functions in `replayer.go`: document exactly what they check and what they return.

Read `internal/audit/async_ledger.go` and `db_writer.go` to understand the async and persistent variants. Document the `LogAction()` function's SQL and the schema of the `alcoa_actions` Postgres table.

Read `pkg/models/audit.go` to document the complete `AuditEntry` struct and all JSON field tags.

Read `proto/swarm.proto` to document the `AuditConfirmation` message, noting which ALCOA+ attributes appear as explicit boolean fields (`attributable`, `legible`, `contemporaneous`, `original`, `accurate`) and which compliance metadata fields exist (`performed_at`, `performed_by`, `method`, `context`).

2.1.2 Step 1.2: Define the Threat Model

Formally specify three attacker tiers:

- **Tier A—Application-level attacker:** Can invoke `Record()` with arbitrary field values but cannot modify stored entries after they are written. Cannot access the database directly.
- **Tier B—Database-level attacker:** Can read and write the Postgres `alcoa_actions` table directly. Knows the genesis hash. Can reconstruct an entire chain from genesis.

- **Tier C—Cryptographic attacker:** Can find collisions or preimages in BLAKE2b-256 (used only for bounding assumptions; we assume this is infeasible).

For each tier, document what the hash chain can and cannot detect. The critical insight for the paper is that Tier B attackers can reconstruct valid chains, which motivates the external anchoring discussed in Phase 2.

2.1.3 Step 1.3: Property-by-Property Classification

For each of the 9 ALCOA+ attributes, perform the following analysis:

1. **State the FDA's definition** of the attribute, citing the relevant FDA guidance document (FDA Guidance for Industry: Data Integrity and Compliance With Drug CGMP, April 2016) and 21 CFR Part 11 subpart text.
2. **Map to vaos-kernel mechanisms:** Identify which code components, data fields, or architectural decisions address this attribute. Reference specific lines of code, struct fields, or protocol messages.
3. **Classify the guarantee level** into one of three tiers:
    - **Tier 1—Cryptographically enforced:** Violation requires breaking a cryptographic assumption (collision resistance of BLAKE2b-256).
    - **Tier 2—Architecturally supported under trust assumptions:** The mechanism works correctly assuming specified components behave honestly (e.g., server clock is correct, credential private keys are not compromised).
    - **Tier 3—Unenforceable by audit log alone:** The property requires external validation beyond what any append-only log can provide.
4. **Document the precise trust assumption** required for Tier 2 properties.

The expected classification (to be verified during analysis):

2.1.4 Step 1.4: Cross-System Comparison Data Collection

For Certificate Transparency (RFC 6962, RFC 9162), collect:

- Read RFC 9162 (the current CT specification) Sections 2–4, documenting the Merkle tree structure, Signed Tree Heads (STHs), and the gossip protocol.
- Document what properties CT's gossip mechanism enforces (log consistency via STH comparison across monitors) and what it does not (it does not attest that certificates were legitimately issued—only that they are included in the log).

For Trillian (the production CT implementation):

- Clone `https://github.com/google/trillian` and read the sparse Merkle tree implementation. Document the inclusion proof mechanism.
- Note that Trillian provides no third-party action attestation—the accuracy gap exists identically.

For Hyperledger Fabric:

- Read the Fabric documentation on endorsement policies (Fabric v2.5 documentation, "Endorsement policies" section).
- Document the architectural analogy: Fabric's `Endorsement` collection is structurally similar to witness cosigning, but designed for business logic validation (checking that a transaction meets chaincode policy), not regulatory data integrity.

ALCOA+ Attribute	Tier	Rationale
Attributable	2	Requires AgentID credential security; `AuditEntry.AgentID` and `AuditConfirmation.performed_by` encode attribution, but a compromised key allows impersonation
Legible	3	Machine-readable JSON ≠ human-readable across regulatory retention periods (decades); format migration is an organizational problem
Contemporaneous	2	`Record()` auto-timestamps in UTC, but requires honest server clock; 60-second JWT window bounds, but does not eliminate, clock skew concerns
Original	1	Hash chain detects post-hoc modification via `VerifyChain()`; collision resistance of BLAKE2b-256 makes tampering detectable
Accurate	3	**Critical finding:** Logging that action X occurred does not prove X occurred; no mechanism in the current implementation validates action reality
Complete	2	Assuming `Record()` is the sole entry path, all actions pass through the chain; but nothing prevents bypassing the API
Consistent	1	Append-only hash chain enforces chronological consistency; reordering breaks the chain
Enduring	2	Postgres persistence provides durability, but backup/archival strategy is an operational concern
Available	3	Single Postgres instance does not satisfy FDA availability requirements; requires replication/backup infrastructure

Table 3: Expected ALCOA+ property classification

For W3C Verifiable Credentials and C2PA:

- Read the W3C Verifiable Credentials Data Model v2.0 (W3C Recommendation) and the C2PA specification v2.0.
- Document that neither specification references ALCOA+, FDA, or 21 CFR Part 11.

Create a comparison table with rows for each ALCOA+ attribute and columns for each system (vaoskernel, CT/Trillian, Hyperledger Fabric, W3C VC, C2PA), showing which tier each attribute achieves in each system.

2.2 Phase 2: Witness Cosigning Protocol Design and Formal Analysis

2.2.1 Step 2.1: Protocol Specification

Write a detailed protocol specification in the style of an IETF Internet-Draft. The protocol modifies the existing `Record()` flow:

```
Protocol WITNESS-COSIGN:

Parameters:
  W: total number of witnesses
```

```
  T: threshold of witness signatures required (T <= W)
  Hash: BLAKE2b-256
  Sig: signature scheme (Ed25519 for software witnesses,
       FIDO2/WebAuthn for human witnesses)

Pre-conditions:
  Agent holds valid intent-scoped JWT credential
  Witness identities and public keys are known to the ledger

Steps:

1. AGENT -> LEDGER:
   Agent invokes Record() with AuditEntry fields populated
   (AgentID, IntentFingerprint, Action, Component, Status, Details)
   Hash is NOT yet computed. Entry is in "pending" state.

2. LEDGER -> WITNESSES:
   For each witness w_i in {w_1, ..., w_W}:
     Ledger dispatches canonical_fields_n to w_i

3. WITNESS VERIFICATION (domain-specific):
   Each w_i independently verifies the action.

4. WITNESS -> LEDGER:
   Each verifying witness w_i returns:
     sigma_i = Sign_{w_i}(canonical_fields_n || w_i.id || timestamp_i)

5. LEDGER AGGREGATION:
   Collect signatures until |{sigma_i}| >= T
   Set AuditEntry.Attestation = Aggregate(sigma_{i_1}, ..., sigma_{i_K})

6. LEDGER CHAINING:
   Compute H_n = BLAKE2b(canonical_fields_n || Attestation_n || H_{n-1})
   Assign H_n as AuditEntry.ID
   Append entry to chain
   Publish H_n to external anchor (see Step 7)

7. EXTERNAL ANCHORING:
   H_n is published to one or more external media:
     - Notarization service (e.g., RFC 3161 timestamp authority)
     - Independent log service
     - Plant historian (manufacturing domain)
     - Blockchain anchoring service

Timeout handling:
  If T signatures are not collected within the JWT validity
  period (60 seconds), the entry is discarded and an error
  is returned. No hash is computed; the chain is unmodified.
```

2.2.2 Step 2.2: Formal Security Game

Define a security game for the (n, t)-threshold accuracy property:

```
Game ACCURACY_{A, n, t}:
```

```
Setup:
  - Initialize ledger with genesis hash
  - Register n witnesses with key pairs (pk_i, sk_i)
  - Adversary A selects up to t-1 witnesses to corrupt
    A receives {sk_i : i in corrupted set}
  - A receives oracle access to Record() for honest witnesses

Challenge:
  - A submits an action (agent_id, action, component) to Record()
  - Honest witnesses verify and sign (or refuse if action did
    not occur -- oracle determines ground truth)
  - A may instruct corrupted witnesses to sign or refuse arbitrarily

Win condition:
  A wins if the ledger accepts an entry where:
  (a) The declared action did NOT actually occur (per oracle), AND
  (b) The entry has >= T valid witness signatures

Advantage:
  Adv^{accuracy}_{A,n,t} = Pr[A wins]

Claim:
  For any PPT adversary A corrupting < t witnesses:
  Adv^{accuracy}_{A,n,t} is negligible (reduction to
  signature unforgeability of the honest witnesses' scheme)
```

Write the reduction proof: if A wins the game, then at least one honest witness's signature was forged (since fewer than t corrupted witnesses cannot meet the threshold, and honest witnesses only sign actions that actually occurred). This reduces to the EUF-CMA security of the signature scheme.

2.2.3 Step 2.3: External Anchor Security Analysis

Formally analyze the external anchoring mechanism against Tier B attackers:

- Define the anchor model: the most recent chain hash H_n is published to an append-only medium that the Tier B attacker cannot modify.
- Show that a Tier B attacker who rewrites entries $k+1, \ldots, n$ must produce a chain ending in H_n (the anchored value), which requires finding a preimage under BLAKE2b-256—infeasible by assumption.
- Compare with CT's approach: CT anchors Merkle tree roots via signed tree heads gossiped to independent monitors. The approach is structurally analogous but CT monitors only verify log consistency, not action accuracy.

2.2.4 Step 2.4: Domain Instantiation Specifications

For each of the three domains, write a concrete instantiation:

Clinical Trial AI:

- Witness: Clinician operator
- Authentication: FIDO2/WebAuthn (satisfies §11.200 two-component: authenticator device + biometric/PIN)
- Signature: FIDO2 assertion signature over `(canonical_fields ‖ witness_id ‖ timestamp)`

- Threshold: 1-of-1 (single clinician countersigns)
- External anchor: Hospital EHR system stores chain hash in an immutable patient record

Lab Automation:

- Witness: PLC/sensor system
- Authentication: Hardware-backed key (TPM or secure element)
- Signature: Ed25519 signature over `(canonical_fields ‖ sensor_reading_hash ‖ witness_id ‖ timestamp)`
- Threshold: Configurable (e.g., 2-of-3 sensors for critical operations)
- External anchor: Laboratory information management system (LIMS)

Pharmaceutical Manufacturing:

- Witness: DCS/SCADA system
- Authentication: Hardware security module (HSM)
- Signature: HSM-backed Ed25519 or ECDSA signature
- Threshold: 1-of-1 for routine operations, 2-of-3 for batch release
- External anchor: Plant historian system (OSIsoft PI or equivalent), which maintains its own append-only record

For each instantiation, document the exact signature payload format, the verification procedure, and how it maps to specific 21 CFR Part 11 sections.

2.3 Phase 3: Quantitative Overhead Analysis

2.3.1 Step 3.1: Reproduce Existing Benchmarks

Clone vaos-kernel and run the existing benchmark suite. Execute:

```
go test -bench=Benchmark -benchmem -count=10 ./...
```

This produces measurements for the four configurations:

- Baseline (no attestation): target ∼51,005 req/s, p99 ∼0.70 ms
- Sync + ALCOA+ attestation: target ∼50,774 req/s, p99 ∼1.69 ms
- Async attestation: target ∼32,827 req/s, p99 ∼87.86 ms
- Sync + Postgres: target ∼20,329 req/s, p99 ∼622 ms

Validate that the reproduced numbers are within 10% of the reported values.

2.3.2 Step 3.2: Witness Cosigning Overhead Model

Since the witness cosigning protocol is not yet implemented, model the overhead analytically:

Cryptographic overhead (per witness signature):

- Ed25519 signing: expected ∼50–100 μs per signature on modern hardware
- Ed25519 verification: ∼70–150 μs per verification
- BLS12-381 aggregation (if used for multi-signature aggregation): expected ∼1–3 ms for aggregation of 5 signatures
- BLAKE2b-256 hash computation over the extended payload: negligible (< 10 μs for payloads < 10 KB)

Network overhead:

- Local witness (same rack, HSM): model as loopback + IPC, ∼0.3–0.5 ms RTT.
- Same-datacenter witness: model as ∼0.5–5 ms RTT.
- Cross-datacenter witness (not recommended for real-time, but relevant for batch): model as ∼10–50 ms RTT.

Threshold overhead:

- For a T-of-W threshold, the latency is dominated by the T-th fastest witness response (assuming parallel dispatch).
- Model as: latency = max(witness_latency$_i$ for $i \in$ fastest T witnesses) + aggregation_time
- For 1-of-1 local: ∼0.5 ms (network) + ∼0.1 ms (crypto) + ∼0.01 ms (hash) ≈ 0.6 ms
- For 3-of-5 mixed (2 local, 3 same-DC): ∼5 ms (3rd fastest witness) + ∼0.3 ms (aggregation) ≈ 5.3 ms

2.3.3 Step 3.3: Composite Overhead Calculation

Compute the total overhead for each configuration by adding witness overhead to the existing baselines:

```
witness_sync_latency = baseline_sync_latency + witness_latency + crypto_overhead
witness_postgres_latency = baseline_postgres_latency + witness_latency + crypto_overhead
```

Since Postgres write latency (622 ms) dominates over witness overhead (< 5 ms for local/same-DC), the relative overhead of witness cosigning is $< 1\%$ in Postgres-backed configurations. This is a key finding.

Create overhead comparison tables:

1. **In-memory configurations:** Show that witness cosigning adds 0.5–80 ms depending on witness topology, all within the 1-second GxP real-time threshold.
2. **Postgres-backed configurations:** Show that witness cosigning adds negligible overhead relative to the 622 ms storage latency.
3. **Throughput analysis:** Estimate throughput impact. For sync + ALCOA+ at 50,774 req/s, adding 0.6 ms local witness reduces throughput to approximately $1/(1/50774+0.0006) \approx 47{,}000$ req/s (roughly 7% reduction). For Postgres at 20,329 req/s, the impact is negligible (∼0.1% reduction).

2.3.4 Step 3.4: Sensitivity Analysis

Vary key parameters and document the impact:

- Number of witnesses W: 1, 3, 5, 7
- Threshold T: 1, 2, 3, 5
- Witness location: local (same rack), same-DC, cross-DC
- Payload size: small (typical AuditEntry ∼500 bytes) vs. large (with extensive Details map ∼5 KB)

Present results as contour plots showing latency as a function of (W, T) for each witness location.

2.4 Phase 4: Regulatory Gap Analysis

2.4.1 Step 4.1: Systematic CFR Section Mapping

Read the following regulatory documents in full:

- 21 CFR Part 11, Subpart B (Electronic Records): §§11.10, 11.30, 11.50, 11.70, 11.100
- 21 CFR Part 11, Subpart C (Electronic Signatures): §§11.100, 11.200, 11.300
- EU Annex 11 (Computerised Systems): all sections
- FDA Guidance for Industry: Data Integrity and Compliance with Drug CGMP (April 2016)
- WHO Technical Report Series, No. 996, Annex 5: Guidance on Good Data and Record Management Practices

For each section, extract:

1. The requirement text (verbatim)
2. The intent of the requirement (from FDA commentary or guidance)
3. Whether vaos-kernel's current implementation satisfies it, partially satisfies it, or does not satisfy it
4. If partially or not satisfied: whether witness cosigning resolves the gap, partially resolves it, or does not address it (requiring organizational controls)

2.4.2 Step 4.2: Gap Classification

Classify each identified gap into three categories:

- **Technical gap, resolved by witness cosigning:** The protocol directly addresses the requirement.
- **Technical gap, requires additional engineering:** Not addressed by witness cosigning but solvable with code changes (e.g., export-to-standard-format).
- **Organizational gap:** Cannot be solved with code; requires process, documentation, or infrastructure (e.g., validation lifecycle documentation per §11.10(a)).

For the paper, we focus on six specific gaps (as outlined in the idea document):

1. No validation lifecycle documentation (§11.10(a)) → Organizational
2. No export-to-standard-format mechanism (§11.10(b)) → Additional engineering
3. 60-second JWT ≠ full retention period (§11.10(c)) → Partially addressed

4. No signature meaning attribution (§11.50) → Resolved by witness cosigning
5. Intent fingerprint ≠ electronic signature (§11.70) → Resolved by witness cosigning (FIDO2)
6. No two-component signing (§11.200) → Resolved by witness cosigning (FIDO2)

For each gap, write a 2–3 paragraph analysis with:

- The exact CFR citation and requirement
- How the gap manifests in vaos-kernel
- How witness cosigning addresses it (if applicable), with specific reference to the protocol steps from Phase 2
- If not addressed by witness cosigning: what organizational control or engineering change is needed

2.4.3 Step 4.3: Constructive Compliance Roadmap

Synthesize the gap analysis into a compliance roadmap that a practitioner could follow:

1. **Deploy witness cosigning** → Resolves gaps 4, 5, 6 (signature-related requirements)
2. **Implement archival storage** → Partially resolves gap 3 (retention)
3. **Build export pipeline** → Resolves gap 2 (standard format)
4. **Establish validation lifecycle** → Resolves gap 1 (organizational; requires SOPs, IQ/OQ/PQ documentation)

This roadmap is included as a practical contribution, not as future work—it is part of the gap analysis itself.

2.5 Phase 5: Comparative Analysis with Existing Systems

2.5.1 Step 5.1: Structured Comparison Framework

Define a comparison framework with the following dimensions:

- **System identity:** Name, specification reference, deployment context
- **Audit mechanism:** Hash chain / Merkle tree / blockchain / other
- **Tamper detection method:** Chain verification / Merkle proof / consensus
- **External anchoring:** Yes/no; mechanism if yes
- **Third-party attestation:** Can external parties attest to the accuracy of logged entries?
- **ALCOA+ awareness:** Does the system explicitly reference or implement ALCOA+ concepts?
- **Regulatory applicability:** Has the system been used in FDA-regulated contexts?
- **Accuracy gap:** Does the system distinguish between "logged X" and "X happened"?

2.5.2 Step 5.2: System-by-System Analysis

Apply the framework to each system:

Certificate Transparency (RFC 6962/9162): Read RFC 9162 Sections 2–6 in detail. Document the Merkle tree structure: each entry is a certificate, leaves are hashed, internal nodes are $H(\text{left}\|\text{right})$, root is published in a Signed Tree Head (STH). Document the gossip protocol: monitors exchange STHs to detect log misbehavior. Analyze the accuracy gap: a CT log certifies that a certificate *was logged*, not that it *was legitimately issued*. Monitors check inclusion, not legitimacy. ALCOA+ mapping: Consistent (Tier 1, Merkle tree), Original (Tier 1, tamper-evident), but Accurate (Tier 3, no attestation of legitimacy).

Trillian: Read the Trillian source code or documentation for the personality API and tree storage. Document that Trillian is a generic Merkle tree log—it provides the same guarantees as CT but is infrastructure, not policy. Same accuracy gap: Trillian does not validate the semantic content of entries.

Hyperledger Fabric: Read the Fabric documentation on the transaction flow: proposal → endorsement → ordering → validation → commit. Document the endorsement policy mechanism: specified peers must sign the transaction proposal response before the transaction is committed. This is architecturally similar to witness cosigning. Key difference: Fabric endorsement validates *business logic* (does the transaction conform to chaincode rules?), not *regulatory data integrity* (is this entry ALCOA+ compliant?). Analyze whether Fabric endorsement policies could be adapted for ALCOA+ compliance: argue yes, with specific policy configurations. Document that Fabric's block structure provides Tier 1 consistency and originality guarantees (via hash chaining of blocks), similar to vaos-kernel.

W3C Verifiable Credentials (VC) / C2PA: Read the W3C VC Data Model v2.0 specification, focusing on the proof mechanism and credential status. Document that VC provides provenance (who issued it) and authenticity (cryptographic proof of issuance) but does not address ALCOA+. Read the C2PA specification, focusing on assertions and ingredient bindings. Document that C2PA addresses media provenance but does not engage with FDA regulatory requirements. Note: neither specification has any reference to ALCOA+, 21 CFR Part 11, or FDA in their normative text.

2.5.3 Step 5.3: Synthesis and Generalization

Create a unified comparison table:

Property	vaos-kernel	CT/Trillian	HF	W3C VC	C2PA	+ Witness
Attributable	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2
Legible	Tier 3	Tier 3	Tier 3	Tier 3	Tier 3	Tier 3
Contemporaneous	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2
Original	Tier 1	Tier 1	Tier 1	Tier 2	Tier 2	Tier 1
Accurate	Tier 3	Tier 3	Tier 3	Tier 3	Tier 3	Tier 1*
Complete	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2
Consistent	Tier 1	Tier 1	Tier 1	N/A	N/A	Tier 1
Enduring	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2	Tier 2
Available	Tier 3	Tier 2	Tier 2	Tier 3	Tier 3	Tier 2
Ext. anchoring	No	Yes	Yes	No	No	Yes

Table 4: Cross-system ALCOA+ comparison. HF = Hyperledger Fabric. *Tier 1 for Accurate under the (n, t)-threshold trust assumption.

Write the accompanying analysis narrative arguing that:

1. The accuracy gap is universal across all existing systems.
2. Witness cosigning is the first mechanism to elevate Accuracy to Tier 1 (under explicit trust assumptions).
3. The external anchor problem is addressed by CT via gossip and by Fabric via consensus; witness cosigning provides a third approach via notarization/HSM publishing, which is more suitable for single-organization regulated deployments where gossip networks are impractical.
4. Hyperledger Fabric's endorsement policies are the closest existing analog to witness cosigning, and the paper's protocol could be adapted as a Fabric chaincode endorsement policy.

2.6 Phase 6: Manuscript Assembly

2.6.1 Step 6.1: Paper Structure

Organize the manuscript as follows (targeting 6–8 pages in IEEE or ACM format):

1. **Introduction** (1 page): Problem statement, the accuracy gap, contribution summary
2. **Background** (1 page): ALCOA+ framework, 21 CFR Part 11, vaos-kernel architecture
3. **Classification of ALCOA+ Properties** (1.5 pages): Phase 1 results, the three-tier classification, the database-level attacker threat
4. **Witness Cosigning Protocol** (2 pages): Phase 2 results, protocol specification, security game, domain instantiations
5. **Overhead Analysis** (1 page): Phase 3 results, quantitative comparison tables
6. **Regulatory Gap Analysis** (1 page): Phase 4 results, the compliance roadmap
7. **Related Work and Comparison** (0.5 pages): Phase 5 results, comparison table
8. **Conclusion** (0.5 pages): Summary of contributions

2.6.2 Step 6.2: Figure and Table Preparation

Create the following artifacts:

- **Figure 1:** Architecture diagram showing the current vaos-kernel `Record()` flow vs. the witness cosigning flow (2-panel figure).
- **Figure 2:** Threat model diagram showing Tier A, B, C attackers and what each can and cannot do.
- **Table 1:** ALCOA+ property classification (9 rows × 4 columns: Property, Tier, vaos-kernel Mechanism, Trust Assumption).
- **Table 2:** Witness cosigning overhead comparison (from Phase 3).
- **Table 3:** Regulatory gap analysis (from Phase 4).
- **Table 4:** Cross-system comparison (from Phase 5).
- **Figure 3:** Latency contour plot (from Phase 3 sensitivity analysis).
- **Listing 1:** Protocol pseudocode (from Phase 2, Step 2.1).

2.6.3 Step 6.3: Writing Assignments

Write sections in this order:

1. First: Section 3 (Classification)—this is the analytical foundation.
2. Second: Section 4 (Protocol)—this is the core contribution.
3. Third: Section 5 (Overhead)—this is quantitative support.
4. Fourth: Section 6 (Regulatory)—this connects to the application domain.
5. Fifth: Section 7 (Related Work)—this positions the contribution.
6. Sixth: Section 2 (Background)—write this after the technical sections to ensure consistency.

7. Last: Section 1 (Introduction) and Section 8 (Conclusion)—write these last to ensure they accurately summarize the paper.

All sections should reference specific code artifacts from vaos-kernel (file names, function names, struct fields) to maintain the paper's grounding in a concrete implementation. Regulatory citations should use the standard CFR format (e.g., "21 CFR § 11.200(a)").

3 Results

3.1 ALCOA+ Attribute-by-Attribute Mapping

ALCOA+ Attribute	21 CFR Part 11 Requirement	vaos-kernel Mechanism	Implementation Evidence
Attributable	Unique user ID, secure authentication (§11.10(d))	`AuditEntry.AgentID` enforced non-empty + 60s JWT with agent_id claim + `AuditConfirmation.performed_by`	`ledger.go:55-63` validates AgentID; JWT binds identity to credential
Legible	Human-readable, retrievable throughout retention (§11.10(b))	JSON serialization via `json.Marshal` + `io.Writer` output + Postgres `alcoa_actions` table	`ledger.go:90-91` writes JSON; `db_writer.go:23-59` persists to SQL
Contemporaneous	Created at time of activity (§11.10(e))	`time.Now().UTC()` auto-timestamp + 60s hard JWT TTL + `performed_at` Unix timestamp	`ledger.go:69-71` stamps on record; JWT verifier enforces `exp-iat==60s`
Original	First recording, unaltered (§11.10(a))	Append-only ledger (no Update/Delete) + BLAKE2b hash chain $H_n = \text{BLAKE2b}(\text{fields}_n \| H_{n-1})$ + `VerifyChain()` tamper detection	`ledger.go:74-82` computes chain; `replayer.go:19-75` independently verifies
Accurate	Error-free, reflects actual activity (§11.10(a))	Deterministic BLAKE2b-256 intent fingerprint + canonical JSON field ordering + required field validation	`ledger.go:127-154` canonical attestation; `ledger.go:55-63` rejects incomplete entries
Complete	All data present including repeats (§11.10(a))	`Details` map captures arbitrary metadata + chain gap detection via `VerifyChain()` + `context` map in gRPC	`audit.go:14` Details field; `swarm.proto:116` context map
Consistent	Cross-system alignment (§11.10(a))	Hash chain enforces strict ordering + UTC timestamps + nanosecond-precision IDs	`ledger.go:66-68` ID generation; `replayer.go:35-54` cross-system verification
Enduring	Maintained for retention period (§11.10(c))	In-memory ledger (50,774 RPS, 100K entry cap) + Postgres `DBWriter` for durable storage + JSON forward-compatible format	`db_writer.go:10-59` persistence; `ledger.go:35-36` configurable capacity
Available	Accessible for review and audit (§11.10(b))	`Entries()` thread-safe copy + SQL queries on `alcoa_actions` + gRPC `ConfirmAudit` RPC + `Replay()` bulk export	`ledger.go:119-125` read access; `swarm.proto:18-19` network access

Table 5: ALCOA+ attribute-by-attribute mapping

3.2 Key Architectural Findings

3.2.1 Hash Chain as Tamper Evidence

The BLAKE2b-256 hash chain ($H_n = \text{BLAKE2b}(\text{canonical\_fields}_n \| H_{n-1})$) provides the mathematical guarantee required by 21 CFR Part 11 §11.10(a) for ensuring records are not altered. This is functionally equivalent to blockchain-style immutability but without the consensus overhead—appropriate for single-agent audit trails where the recording system is trusted.

3.2.2 Performance Under Regulatory Load

Sync ALCOA+ attestation adds only 0.5% overhead (50,774 vs 51,005 req/s). This demonstrates that regulatory compliance need not compromise system performance. The sub-microsecond BLAKE2b computation is faster than the goroutine coordination in the async path.

3.2.3 Dual-Layer Storage

The in-memory ledger handles hot-path operations while `DBWriter` provides 21 CFR Part 11 §11.10(c) retention compliance via Postgres. This separation allows performance-critical paths to maintain throughput while meeting durability requirements.

3.2.4 gRPC ALCOA+ Booleans

The `AuditConfirmation` proto message includes explicit boolean fields for each core ALCOA attribute (`attributable`, `legible`, `contemporaneous`, `original`, `accurate`). This allows the calling system to self-attest compliance at the API boundary, creating a machine-readable compliance signal.

3.2.5 Independent Verification

`Replay()` enables third-party auditors to verify chain integrity without access to the original system. The well-known genesis hash and deterministic `attestChained()` function mean any party can reproduce the chain from raw entries.

3.3 Gap Analysis

Attribute	Coverage	Gap
Attributable	Full	Agent identity relies on caller-provided AgentID; no mutual TLS or certificate-based agent auth
Legible	Full	No built-in PDF/print export for audit reports
Contemporaneous	Full	Clock skew between distributed agents not addressed
Original	Full	In-memory ledger loses data on restart (mitigated by Postgres mode)
Accurate	Full	No schema validation on `Details` map contents
Complete	Partial	No explicit "repeat/reanalysis" marker field
Consistent	Full	Single-process chain; multi-node consistency requires external coordination
Enduring	Full (with Postgres)	In-memory mode alone does not meet retention requirements
Available	Full	No built-in access control on `Entries()`—any caller can read all entries

Table 6: Gap analysis by ALCOA+ attribute

3.4 Regulatory Applicability

This mapping demonstrates applicability to:

1. **Pharmaceutical manufacturing**—AI agents controlling batch processes must produce 21 CFR Part 11 compliant records

2. **Clinical trials**—AI agents processing patient data must maintain attributable, contemporaneous audit trails

3. **Medical device software**—AI agents embedded in SaMD (Software as a Medical Device) must produce tamper-evident records per FDA guidance

4. **Financial services**—SEC Rule 17a-4 and FINRA requirements share structural similarity with ALCOA+ for electronic record retention

4 Conclusion

This paper presents a comprehensive mapping between FDA 21 CFR Part 11 ALCOA+ requirements and the hash-chained audit implementation in vaos-kernel. We identify the accuracy gap in hash-chained audit trails and propose witness cosigning as a constructive solution that elevates accuracy from an unenforceable property to a cryptographically guaranteed one under explicit trust assumptions.

The witness cosigning protocol adds negligible overhead (sub-millisecond with local HSM, $\sim$5ms with same-datacenter witnesses) compared to existing storage latency (622ms for Postgres), making it viable for real-time pharmaceutical manufacturing systems subject to GxP requirements.

Future work includes implementing the witness cosigning protocol in vaos-kernel and conducting a formal security analysis of the (n, t)-threshold accuracy property.